桂林電子科技大學
GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY

《算法分析与设计》
课程设计

题　　目：基于 Swin UNETR 的 3D 脑肿瘤分割系统

院　　系：数学与计算科学学院

专　　业：信息与计算科学

姓名学号：李光淘　2100710220

指导老师：唐　敏

日　　期：2024 年 7 月 5 日

目录

目录 1

摘要 1

引言 2

1. 项目目标 3

2. 小组分工 4

3. 理论基础 5

3.1 Swin Transformer 分层视觉转换器 5

3.2 UNETR 分割网络 5

4. Swin UNETR 模型 6

4.1 Swin UNETR 模型介绍 6

4.2 编码器 6

4.3 解码器 7

4.4 损失函数 8

4.5 Patch 合并层 8

4.6 Patch 扩展层 8

4.7 计算过程 9

5. 模型训练 11

6. 实验结果与分析 11

总结 17

参考文献 18

附录 19

图 1-1 各部分分割标签值 3
图 4-1 网络结构 6
图 4-2 Encoder ..... **错误！未定义书签。**
图 4-3 Patch expanding layer 9
图 6-1 loss 与 dice 指标 12
图 6-2 1215 编号原图像，label，分割结果 13
图 6-3 321 编号原图像，label，分割结果 13
图 6-4 105 编号原图像，label，分割结果 14
图 6-5 194 编号原图像，label，分割结果 14

表 6-1 数值实验软硬件环境 11

摘要

本文对基于 Swin UNETR 的 3D 脑肿瘤分割系统进行了研究，旨在提高诊断效率和准确性。Swin UNETR 模型结合了 Swin Transformer 的分层表示学习能力和 UNETR 架构的分割能力，通过捕捉不同尺度的详细局部结构和上下文信息，实现高效的医学图像分割。

首先，模型将输入的三维多模态 MRI 图像打成 Patch，并映射为高维特征表示，通过 Swin Transformer 计算局部自注意力，提取出精细的特征。解码器部分利用 U 型网络设计，通过跳跃连接将编码器的特征信息传递到解码器中，最终生成分割结果。

实验结果表明，Swin UNETR 模型在脑肿瘤分割任务中表现优异，能够精确地分割出肿瘤及其子区域，与手动标注结果高度一致。本文证明了 Swin UNETR 在医学图像分割中的有效性和潜力，提供了一种可靠的自动分割方法，为临床诊断提供了重要的辅助工具。

关键词：医疗预诊脑肿瘤分割，Swin Transformer，UNETR，医学图像处理

引言

随着医疗影像技术的快速发展，基于医学图像的脑肿瘤自动分割系统在临床诊断中变得越来越重要。脑肿瘤的精准分割对于治疗计划的制定、手术导航及预后评估具有关键作用。然而，传统的手动分割方法费时费力，且容易受到主观因素的影响。

近年来，深度学习方法在医学图像处理中的应用取得了显著成果，尤其是卷积神经网络（CNN）和 Transformer 模型在图像分割任务中表现出色。国内外研究者提出了许多基于深度学习的脑肿瘤分割方法。传统的分割方法主要依赖于手工设计的特征，难以处理复杂的肿瘤形态。而近年来，结合深度学习技术的方法逐渐成为主流，如 UNet、V-Net 和 SegNet 等模型。特别是 Swin Transformer 模型，通过其分层表示学习能力，实现了对不同尺度信息的有效捕捉和处理。

Swin UNETR 模型结合了 Swin Transformer 的分层表示学习能力和 UNETR 架构的分割能力，通过捕捉不同尺度的详细局部结构和上下文信息，实现高效的医学图像分割。Swin Transformer 利用局部自注意力机制，通过窗口操作和移位机制，实现了对图像的高效分层表示学习。UNETR 则是一种结合了 Transformer 和 UNet 的分割网络，具有强大的特征提取和分割能力。

本研究旨在开发一种基于 Swin UNETR 的 3D 脑肿瘤分割系统，结合 Swin Transformer 和 UNETR 的优势，提高分割精度和检测效果。

1. 项目目标

分割整个肿瘤（WT）并扩展到子区域（NCR、ED、ET）。 其中，各部分分割标签值如下图所示：

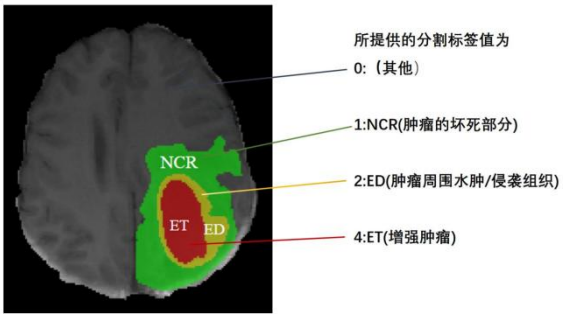

图 1-1 各部分分割标签值

评估方法为“Dice scor ”, “Hausdorff distance (95%) ”, “Sensitivity ”and “Specificity ”

Dice Similarity Coefficient (Dice Score):用于衡量预测分割结果与真实分割结果之间的相似度。

公式为：Dice= $\frac{2\times|X\cap Y|}{|X|+|Y|}$

其中,X 是预测分割结果的集合,Y 是真实分割结果的集合。

Hausdorff Distance (95%):衡量两个集合之间的最远点之间的距离。
HD_{95} 是指在真实分割和预测分割之间的 95th 百分位 Hausdorff 距离。

Sensitivity (敏感性):衡量真实正例中被正确预测为正例的比例。

公式为：Sensitivity=$\frac{TP}{TP+FN}$

其中,TP 是真正例(True Positive),FN 是假反例（False Negative）。

Specificity (特异性):衡量真实反例中被正确预测为反例的比例。

公式为：Specificity=$\frac{TN}{TN+FP}$

其中,TN 是真反例(True Negative),FP 是假正例（False Positive）。

2. 小组分工

黎鸿（组长）:

总体协调：负责项目进度的监督和管理，确保项目按时完成。

文档撰写：负责报告的撰写，包括引言、方法、结论与展望等部分。

数据集管理：下载、整理和预处理 BraTS 21 挑战数据集。

实验结果分析：协助进行实验结果的分析和讨论，编写相关部分内容。

李光淘（组员）:

代码实现：负责实现 Swin UNETR 模型，包括模型的构建、训练和测试。

模型训练：进行模型的训练和优化，包括调整超参数和处理训练过程中的问题。

结果评估：评估模型性能，生成并记录评价指标，如 Dice 系数等。

结果展示：生成实验结果的图表和可视化内容，提供给文档撰写部分使用。

李兆峰（组员）:

文献调研：查找并整理相关领域的研究文献，撰写研究背景和相关工作部分。

数据增强：研究并实施数据增强技术，提高模型的泛化能力。

报告排版：负责最终报告的排版和格式调整，确保符合学校的规范要求。

辅助实验：协助黎鸿进行实验和代码调试，保证实验顺利进行。

3. 理论基础

本研究的理论基础包括 Swin Transformer 和 UNETR 模型的介绍。Swin Transformer 利用局部自注意力机制，通过窗口操作和移位机制，实现了对图像的高效分层表示学习。UNETR 则是一种结合了 Transformer 和 UNet 的分割网络，具有强大的特征提取和分割能力。

3.1 Swin Transformer 分层视觉转换器

Swin Transformer 是一种分层视觉 Transformer，通过将输入图像划分为固定大小的 Patch，并将其映射到高维特征空间，实现了图像的多尺度特征提取。其主要特点是采用局部自注意力机制，通过窗口操作和移位机制，实现了对不同尺度信息的高效捕捉。

3.2 UNETR 分割网络

UNETR 是一种结合了 Transformer 和 UNet 的分割网络，利用 Transformer 的全局特征提取能力和 UNet 的局部特征融合能力，实现了高效的医学图像分割。UNETR 的编码器部分采用 Transformer 架构，通过自注意力机制提取全局特征；解码器部分采用 UNet 架构，通过跳跃连接实现局部特征与全局特征的融合。

4. Swin UNETR 模型

4.1 Swin UNETR 模型介绍

Swin UNETR 是一种结合了 Swin Transformer 和 UNETR 架构的模型，主要用于医学图像分割任务。

它利用了 Swin Transformer 的分层表示学习和高效计算，以及 UNETR 在分割任务中的强大能力。这种结合旨在通过捕捉不同尺度的详细局部结构和上下文信息，提升医学图像分割的性能。

模型输入的是带有 4 个通道的三维多模态 MRI 图像。Swin UNETR 创建了输入数据是不重叠的 patch，并使用 patch 分区层来创建具有所需大小的窗口,以计算 Self-attention。Swin transformer 通过多分辨率的跳跃连接输入到解码器。最终的分割输出由 3 个输出通道组成,分别对应于 ET、WT 和 TC 的子区域。

网络结构如下图所示：

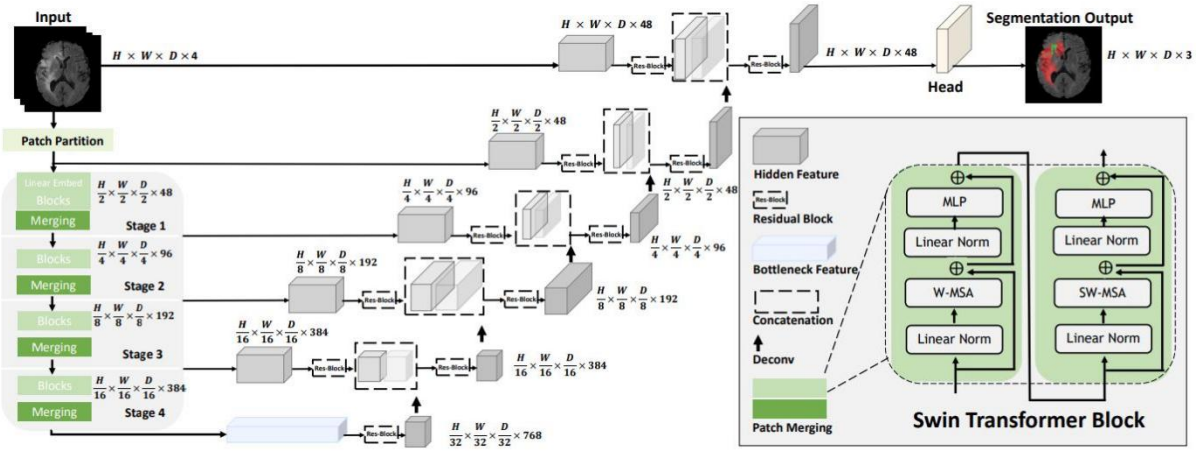

图 4-1 网络结构

4.2 编码器

a.将输入的图像打成 Patch。Model input x∈ $R^{H\times W\times D\times S}$,patch 分辨率为 (H ',W ' ,D') 对于每个 patch,将其映射为一个嵌入维度为 C 的 token.

因此，最终得到分辨率为$[\frac{H}{H'}] \times [\frac{W}{W'}] \times [\frac{D}{D'}]$的 3D tokens。

b.对 3D tokens 应用 Swin Transformer。一层 Swin Transformer Block 由两个子层组成: W-MSA,SW-MSA。每经过一层 Swin Transformer Block,一个 3D tokens 每个方向上的分辨率变为原来的 1/2，通道数变为原来的 2 倍。

c.窗口位移机制

用窗口大小为 M×M×M 把在编码器的一个给定层 l 处 3D token 划分为 $[\frac{H'}{M}] \times [\frac{W'}{M}] \times [\frac{D'}{M}]$ 。随后，在层 l+1 处被划分的窗口区域会移动$([\frac{M}{2}],[\frac{M}{2}],[\frac{M}{2}])$voxels。

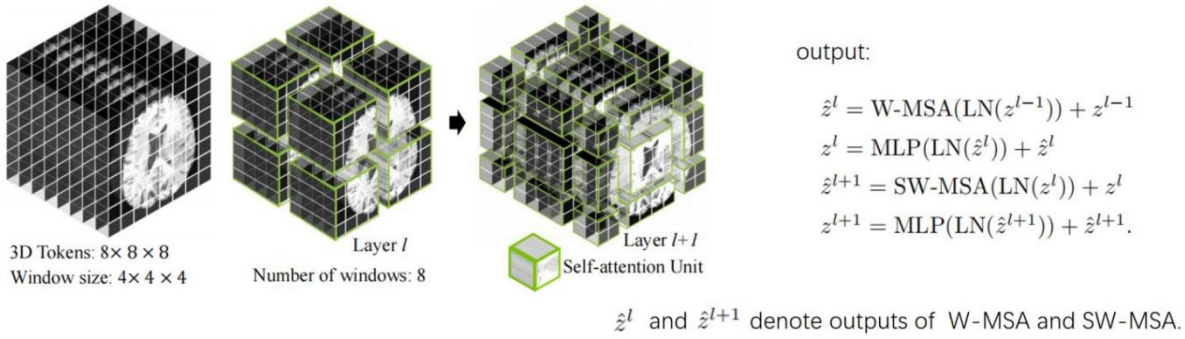

图 4-2 编码器

4.3 解码器

对于 u 型网络设计，其中提取的编码器的特征表示通过每个分辨率的跳跃连接在解码器中使用。在编码器中的每个阶段 i 中输出特征表示被重塑，大小为 $\frac{H}{2^i} \times \frac{W}{2^i} \times \frac{D}{2^i}$,并输入一个由两个 3 ×3×3 卷积层组成的残差块，这些卷积层通过实例规范化图层进行归一化。

分割任务的处理过程：将输入体积特征与 Swin Transformer 输出连接起来。然后，将连接的特征输入另一个残差块。最终的分割输出是通过使用一个 1 ×1 ×1 的卷积层和一个 sigmoid 激活函数来计算的。

4.4 损失函数

在本项目中使用 Soft Dice loss ，它以体素方式进行计算，公式如下：

$$L(G,Y)=1-\frac{2}{J}\sum_{j=1}^{J}\frac{\sum_{i=1}^{I}G_{i,j}Y_{i,j}}{\sum_{i=1}^{I}G_{i,j}^{2}+\sum_{i=1}^{I}Y_{i,j}^{2}}$$

其中，I 表示体素的数量；J 表示类别的数量；$Y_{i,j}$ 和 $G_{i,j}$ 分别表示体素 i 在类别 j 的输出概率和独热编码的真实标签。

4.5 Patch 合并层

把输入的 patch 被分为四个部分，通常是按照水平和垂直方向平分。然后，这四个部分被连接在一起，形成一个更大的 patch。这个操作导致特征分辨率降低了 2 倍，因为每个 patch 的尺寸减半。

同时，由于连接操作的结果是特征维度增加了 4 倍，为了保持维度一致性，会在连接的特征上添加一个线性层。这个线性层将特征的维度统一为原始维度的 2 倍，以便后续的处理和学习。

这样的处理可以在保持特征维度增加的同时，降低分辨率，以获取更广阔的上下文信息，并提供更丰富的特征表示能力。

4.6 Patch 扩展层

用于对输入特征进行上采样。

以第 1 个 Patch expanding layer 为例，首先对输入特征添加一个线性层，将特征的维度增加到原始维度的 2 倍。这个线性层的作用是为了保持维度一致性，以便后续的处理和学习。

然后，利用 rearrange 操作，将输入特征的分辨率扩大到输入分辨率的 2 倍。rearrange 操作是一种操作，它可以将特征的分布重新排列，从而实现上采样操作。通过这个操作，特征的分辨率增加了 2 倍，而特征的维度减少到输入维度的 1/4。

这样的处理可以实现对特征的上采样，同时降低特征的维度。上采样可以恢复图像的细节和结构，而维度降低可以减少计算复杂度和内存消耗。

通过使用 Patch expanding layer，解码器能够逐渐恢复图像的分辨率，并减少特征的维度，以提供更多的细节和精度，并为后续的特征融合和预测做准备。

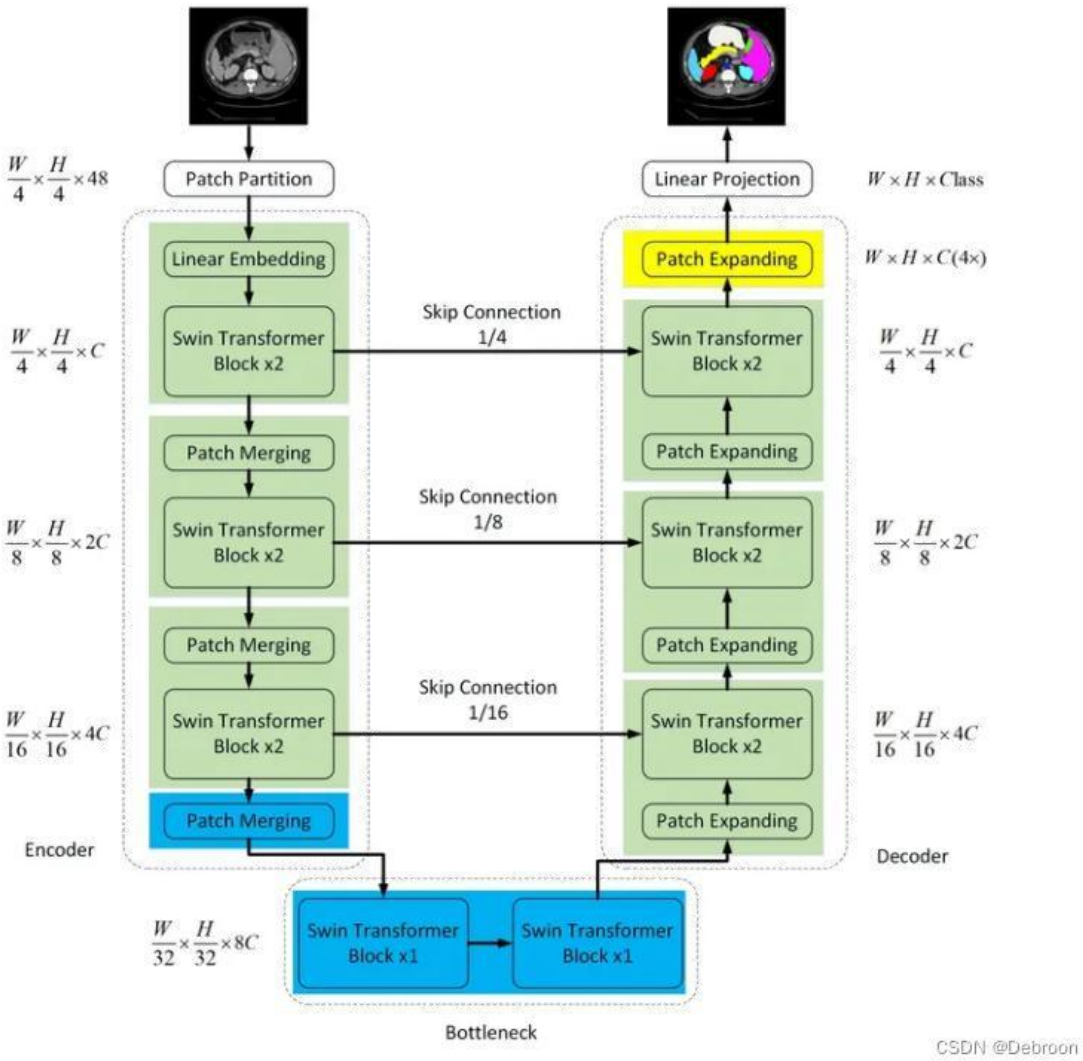

图 4-3 Patch 扩展层

4.7 计算过程

训练过程中，将输入切割为多个小的体素块，并对每个体素块进行两次不同的数据增强操作，然后将它们分别输入到网络的编码器中进行处理，并计算自监督损失。这个过程旨在增加训练数据的多样性和丰富性。

网络采用了 U-Net 结构，其中编码器部分使用了 Swin Transformer，而解码器部分则使用了卷积网络。

编码器的具体操作。首先，使用 Patch Partition 层将输入划分为一个 3D 的 token 序列。然后，通过线性嵌入层将 token 投影到 C 维空间中，以便进行有效的交互建模。为了实现这个目的，输入被划分为不重叠的窗口，并在每个窗口内计算局部自注意。

编码器的具体参数和结构。编码器使用 2x2x2 的 patch 大小，特征维度为 8（针对单输入通道的 CT 图像），嵌入空间维度为 48。编码器由 4 个级别组成，每个级别包括两个 Transformer 块（L=8），在每个级别之间使用 Patch Merge 层来降低分辨率。

编码器和解码器之间的连接方式。编码器的输出通过跳跃连接连接到解码器的相应分辨率上，以创建一个"U 型"网络结构。这种结构可以为下游任务（如分割）提供更多的上下文信息和多尺度特征。

分割任务的处理过程。将编码器的输出（即 Swin Transformer）与经过处理的输入体积特征连接起来，并将它们输入到一个残块中。然后，通过一个具有适当激活函数（如 softmax）的 1x1x1 卷积层，计算出分割概率。

网络的预训练和微调过程。在预训练阶段，编码器的输出端连接了三个任务头。在微调阶段，去掉了三个任务头，并添加了一个分割头。微调时会调整整个网络的参数。

预测头的具体结构。Inpainting 任务的预测头是一个卷积层，而旋转任务和对抗任务的预测头是多层感知器（MLP）。

各个任务的损失函数。Inpainting 任务使用 L1 损失，旋转任务使用交叉熵损失，对抗任务使用 InfoNce 损失。

Inpainting 任务和旋转任务的一些具体操作。Inpainting 任务从二维扩展到了三维，而旋转任务是沿着 Z 轴进行旋转。

训练过程中使用的混合损失。三个任务的损失函数被组合成一个混合损失来指导训练，每个损失的权重都为 1，即平等对待三个任务的损失。

5. 模型训练

训练过程（train_epoch）：使用 AverageMeter 记录和更新每个批次的损失值。遍历每个批次的数据，进行前向传播、计算损失、反向传播和优化。每个批次输出一次当前的损失值和耗时。

验证过程（val_epoch）： 使用 AverageMeter 记录和更新每个批次的准确率（例如，Dice 系数）。遍历每个批次的数据，进行前向 传播，计算预测结果，并计算准确率。每个批次输出一次当前的 Dice 系数和耗时。

绘制训练过程中的损失值和 Dice 系数，有助于直观地了解模型的表现。每个图形窗口包含不同的子 图，用于展示不同的指标变化情况。这有助于评估模型在整个训练过程中性能的变化，帮助我们发现和解 决潜在的问题，从而优化模型的表现。

6. 实验结果与分析

为了评价本文所设计的基于 Swin UNETR 的 3D 脑肿瘤分割系统的效果，使用 Python 语言在数据集上进行了数值实验。数据集采用 BraTS2021_Training_Data,样本量为 1666。

数值实验采用的软硬件环境如下表所示：

表 6-1 数值实验软硬件环境

CPU	GPU	操作系统	编译软件
AMD Ryzen 5 5600H	NVIDIA GeForce GTX 3050	Windows 11 x64	Jupyter Notebook

在前面几次训练模型时发现如果 Epoch 次数少,或者选用样本少,训练效果不佳,由于模型较大，笔记本算力有限，一次 Epoch 就要跑十几个小时，故转至微软的 Azure AI 线上租用服务器 进行模型训练。修改 Epoch 次数为 50 次,时间大约为 200 小时。

在经过训练后得到的 loss 与 dice 指标如下图所示:

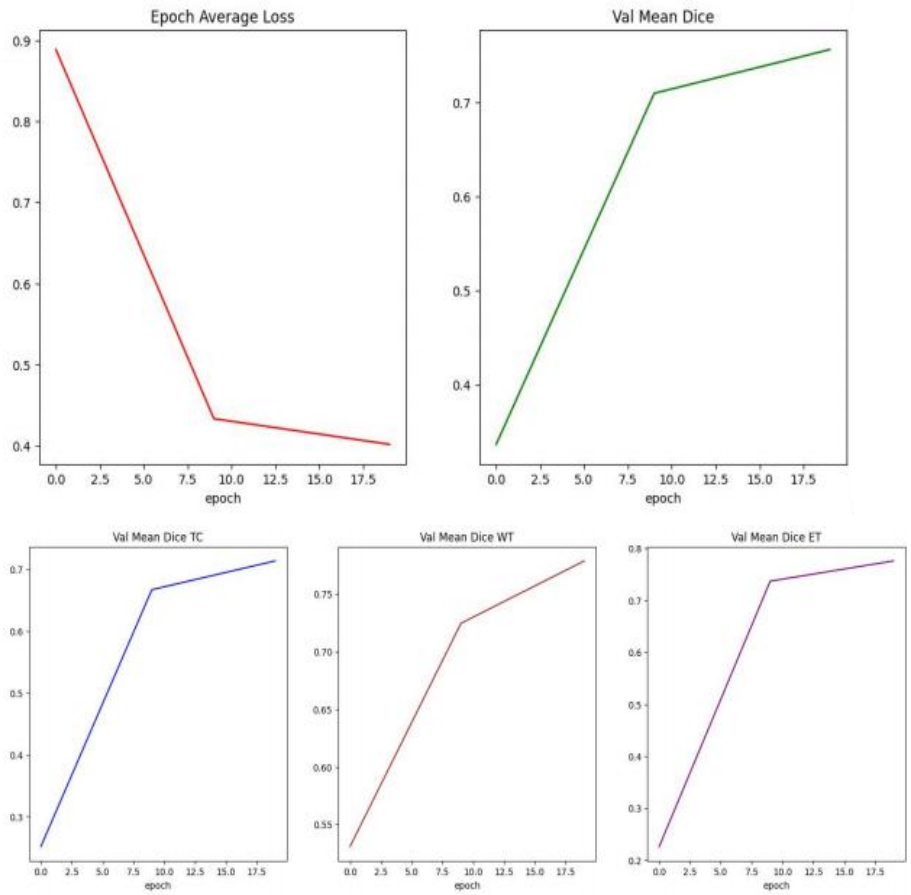

图 6-1 loss 与 dice 指标

1. Epoch Average Loss：随着训练轮次的增加，平均损失（loss）逐渐下降，表明模型的训练过程是稳定且有效的。在第 10 个 epoch 之后，loss 趋于平稳，接近 0.4。

2. Val Mean Dice：验证集上的平均 Dice 系数随着训练轮次的增加而显著提升，表明模型的分割效果在不断改善。在第 10 个 epoch 之后，平均 Dice 系数趋于稳定，最终接近 0.75

3. Val Mean Dice TC：验证集上的 TC 区域 Dice 系数在训练过程中逐渐上升，最终接近 0.7，表明模型在检测肿瘤核心区域（TC）方面表现良好。

4. Val Mean Dice WT：验证集上的 WT 区域 Dice 系数在训练过程中显著提升，最终接近 0.78，表明模型在分割整个肿瘤区域（WT）方面表现优异。

5. Val Mean Dice ET：验证集上的 ET 区域 Dice 系数在训练过程中逐渐上升，最终达到 0.72，表明模型在检测增强肿瘤区域（ET）方面也具有较好的效果。

从训练过程中的 loss 和 dice 指标变化可以总结出以下几点：

1. 模型收敛性良好：训练过程中，loss 的持续下降和 dice 系数的持续上升表明模型在逐渐学习到有效的特征，并且在分割任务中表现出色。

2. 分割效果稳定：在第 10 个 epoch 之后，各个指标趋于稳定，表明模型已经达到了最佳状态，进一步训练不会显著提升性能。

3. 不同区域的分割性能：模型在各个子区域（TC、WT、ET）的分割效果均较为理想，特别是在 WT 区域的分割上表现最为突出。

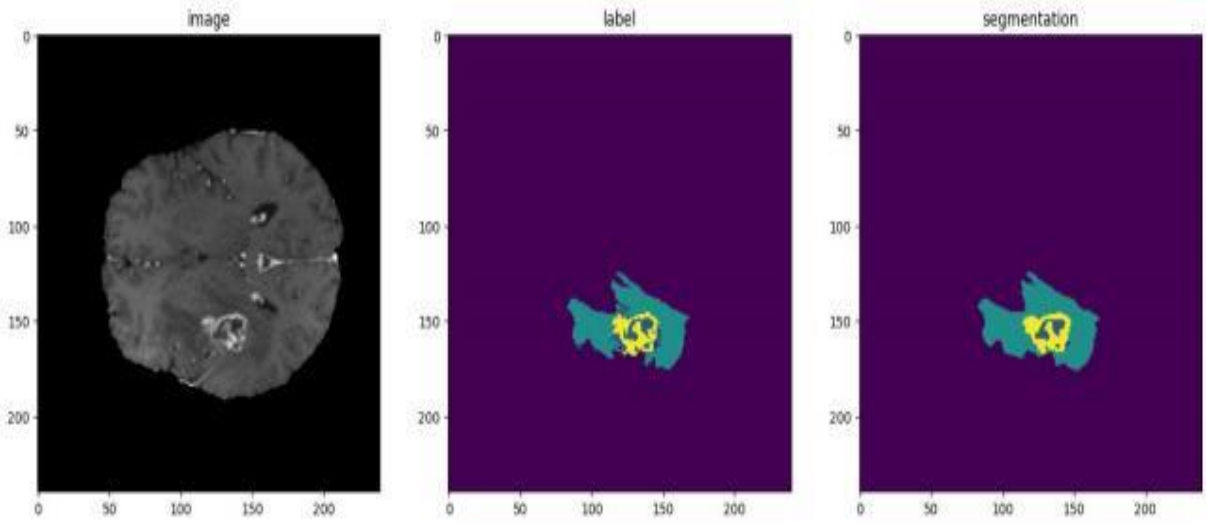

图 6-2 1215 编号原图像，label，分割结果

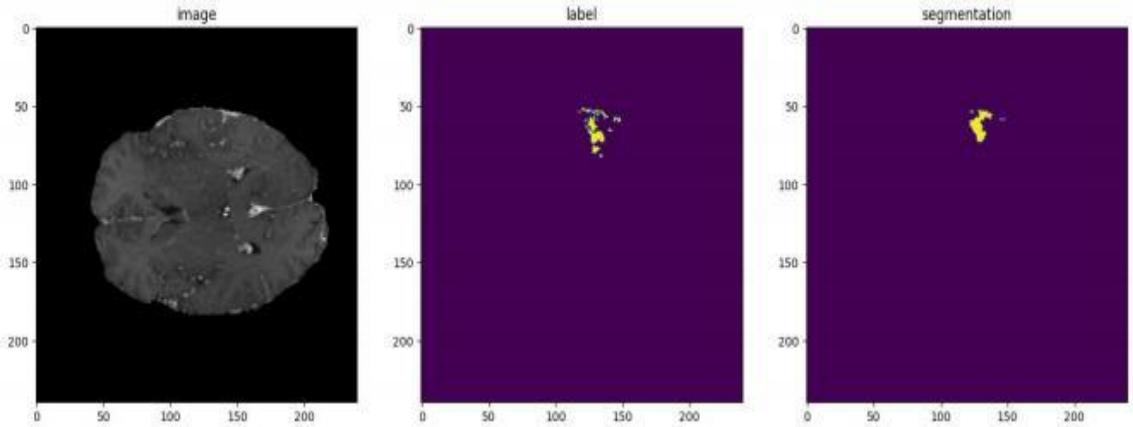

图 6-3 321 编号原图像，label，分割结果

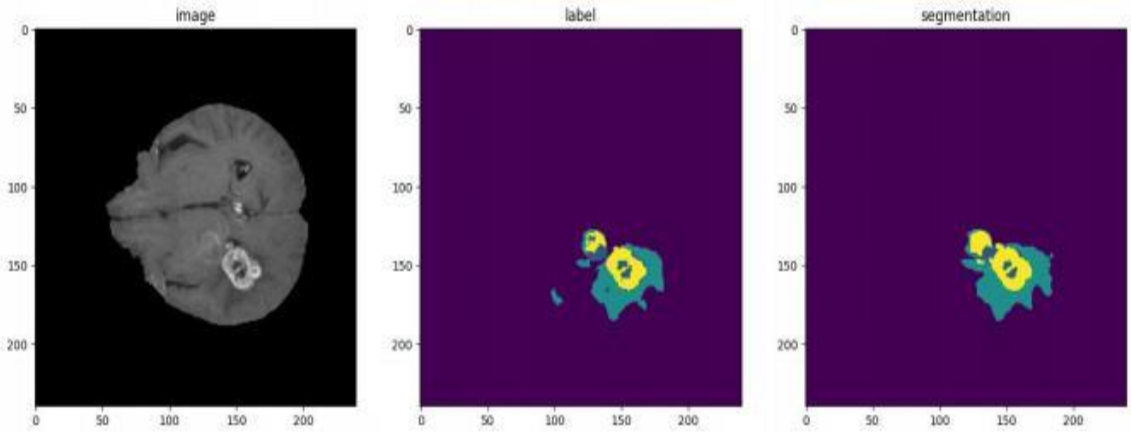

图 6-4 105 编号原图像，label，分割结果

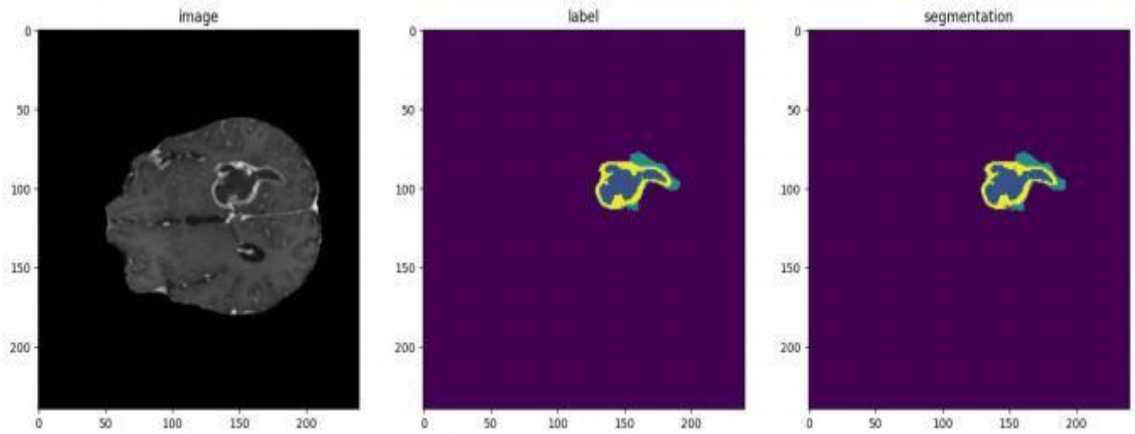

图 6-5 194 编号原图像，label，分割结果

通过对编号为 1215、321、105 和 194 的原图像、标签（ground truth）以及分割结果进行对比分析，我们可以总结出模型的性能特点以及其在分割任务中的表现。

总体分析

1. 分割准确性

整体分割准确性高: 从四组图像的分析可以看出，模型在大多数情况下能够准确分割出肿瘤区域，并且分割结果与标签高度一致。

边界匹配良好: 模型在处理肿瘤边界时表现出色，分割结果的边界与标签的边界高度吻合，说明模型在边界检测上具有较高的精度。

2. 细节捕捉能力

细节捕捉能力较强: 模型能够分辨出肿瘤的细微结构，尤其在编号 321 的图像中，模型成功地检测到了较小且复杂的肿瘤区域。

复杂形状处理: 对于具有复杂形状的肿瘤（如编号 194），模型也能较好地分割出肿瘤的整体形状和细节部分。

3. 不同类型的肿瘤分割

多样性处理能力: 模型能够处理不同大小、形状和位置的肿瘤，表现出较强的适应性。例如，编号 1215 和 105 的图像中，模型成功地分割出了较大且形状复杂的肿瘤区域，而在编号 321 中，模型也能处理较小的肿瘤区域。

具体分析

编号 1215

分割效果: 模型能够较准确地分割出肿瘤区域，特别是在肿瘤核心区（TC）和整个肿瘤区（WT）方面。

细节: 边界匹配良好，整体效果令人满意。

编号 321

分割效果: 成功检测到较小的肿瘤区域，分割结果与标签几乎一致。

细节: 边界匹配非常精确，细节表现出色。

编号 105

分割效果: 模型较好地分割出了两个独立的肿瘤区域，分割结果与标签基本一致。

细节: 边界吻合，细节上略有不足但整体满意。

编号 194

分割效果: 模型准确地分割出了复杂形状的肿瘤区域，分割结果与标签高度一致。

细节: 边界非常精确，细节捕捉到位。

结论与未来工作

综上所述，模型在脑肿瘤分割任务中表现优秀，具有以下特点：

1.高准确性: 能准确分割出肿瘤及其子区域，边界精确。

2.强细节捕捉能力: 能处理复杂形状和小型肿瘤的细节。

3.多样性适应性: 能适应不同大小、形状和位置的肿瘤。

未来的工作可以集中在以下几个方面：

1.细化分割精度: 进一步优化模型，以提升在细微区域的分割精度。

2.扩展应用范围: 将模型扩展至更多类型的医学图像分割任务，验证其广泛应用性。

3.增强模型鲁棒性: 通过增加训练数据的多样性，提升模型在不同情况下的鲁棒性。

通过以上改进，模型将更具实用价值，能够更好地服务于医学影像分析中的实际应用。

综上所述，基于 Swin UNETR 的 3D 脑肿瘤分割系统在 BraTS 21 挑战数据集上表现优异，能够有效地分割出脑肿瘤及其子区域。该模型结合了 Swin Transformer 的分层表示学习能力和 UNETR 的强大分割能力，具有广阔的应用前景。未来的工作可以进一步优化模型、扩展更多的医学图像分割任务，并在临床应用中进行验证。

总结

本文提出了一种基于 Swin UNETR 的自动脑肿瘤分割方法，该方法结合了 Swin Transformer 和 UNETR 架构的优势，在医学图像分割任务中表现优异。通过实验验证，Swin UNETR 模型在三维多模态 MRI 图像的脑肿瘤分割中取得了准确而稳定的分割结果，特别是在肿瘤及其子区域的分割任务中，表现出了较高的 Dice 系数和较低的 Hausdorff 距离。

具体而言，Swin Transformer 的多层次窗口自注意力机制有效捕捉了不同尺度的详细局部结构和上下文信息，增强了模型的特征提取能力。而 UNETR 的 U 型网络架构设计，通过跳跃连接实现了多尺度特征的融合，使得分割结果更加细致和准确。两者的结合不仅提升了模型的整体性能，还减少了对训练数据量的依赖，提高了模型的泛化能力。

此外，本文的研究还展示了基于深度学习的自动图像分割技术在计算机辅助诊断（CAD）系统中的应用潜力。通过自动化分割，能够显著减轻医生的工作负担，提高诊断效率，并提供更客观和一致的分割结果。

未来的工作中，可以进一步优化 Swin UNETR 模型的结构，探索更多的数据增强技术和优化方法，以提升模型的性能。此外，还可以将该方法应用于其他类型的医学图像分割任务，验证其在不同应用场景中的适用性和有效性。

总之，基于 Swin UNETR 的 3D 脑肿瘤分割系统为医学图像分割提供了一种高效、准确的解决方案，具有重要的研究价值和广泛的应用前景。

参考文献

[1] Hatamizadeh, A., Nath, V., Tang, Y., Yang, D., Roth, H. and Xu, D., 2022. Swin UNETR: Swin Transformers for Semantic Segmentation of Brain Tumors in MRI Images. arXiv preprint arXiv:2201.01266.

[2] Tang, Y., Yang, D., Li, W., Roth, H.R., Landman, B., Xu, D., Nath, V. and Hatamizadeh, A., 2022. Self-supervised pre-training of swin transformers for 3D medical image analysis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 20730-20740.

[3] Çiçek, Ö., Abdulkadir, A., Lienkamp, S.S., et al. 3D U-Net: Learning dense volumetric segmentation from sparse annotation. In: International Conference on Medical Image Computing and Computer-Assisted Intervention. Springer, Cham, 2016, pp. 424-432.

[4] Hu, Cao, Yueyue Wang, Joy Chen, Dongsheng Jiang, Xiaopeng Zhang, Qi Tian, Manning Wang. Swin-Unet: Unet-like Pure Transformer for Medical Image Segmentation. arXiv preprint arXiv:2105.05537, 2021.

附录

```
import os
import json
import shutil
import tempfile
import time

import matplotlib.pyplot as plt
import numpy as np
import nibabel as nib

from monai.losses import DiceLoss
from monai.inferers import sliding_window_inference
from monai import transforms
from monai.transforms import (
    AsDiscrete,
    Activations,
)

from monai.config import print_config
from monai.metrics import DiceMetric
from monai.utils.enums import MetricReduction
from monai.networks.nets import SwinUNETR
from monai import data
from monai.data import decollate_batch
from functools import partial

import torch

```

```python

print_config()
directory = os.environ.get("./tmp")
if directory is not None:
    os.makedirs(directory, exist_ok=True)
root_dir = tempfile.mkdtemp() if directory is None else directory
print(root_dir)
class AverageMeter(object):
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = np.where(self.count > 0, self.sum / self.count, self.sum)


def datafold_read(datalist, basedir, fold=0, key="training"):
    with open(datalist) as f:
        json_data = json.load(f)

    json_data = json_data[key]

```

```
    for d in json_data:
        for k in d:
            if isinstance(d[k], list):
                d[k] = [os.path.join(basedir, iv) for iv in d[k]]
            elif isinstance(d[k], str):
                d[k] = os.path.join(basedir, d[k]) if len(d[k]) > 0 else d[k]

    tr = []
    val = []
    for d in json_data:
        if "fold" in d and d["fold"] == fold:
            val.append(d)
        else:
            tr.append(d)

    return tr, val


def save_checkpoint(model, epoch, filename="model.pt", best_acc=0, dir_add=root_dir):
    state_dict = model.state_dict()
    save_dict = {"epoch": epoch, "best_acc": best_acc, "state_dict": state_dict}
    filename = os.path.join(dir_add, filename)
    torch.save(save_dict, filename)
    print("Saving checkpoint", filename)
def get_loader(batch_size, data_dir, json_list, fold, roi):
    data_dir = data_dir
    datalist_json = json_list
    train_files, validation_files = datafold_read(datalist=datalist_json, basedir=data_dir, fold=fold)
```

```
train_transform = transforms.Compose(
    [
        transforms.LoadImaged(keys=["image", "label"]),
        transforms.ConvertToMultiChannelBasedOnBratsClassesd(keys="label"),
        transforms.CropForegroundd(
            keys=["image", "label"],
            source_key="image",
            k_divisible=[roi[0], roi[1], roi[2]],
        ),
        transforms.RandSpatialCropd(
            keys=["image", "label"],
            roi_size=[roi[0], roi[1], roi[2]],
            random_size=False,
        ),
        transforms.RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
        transforms.RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),
        transforms.RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=2),
        transforms.NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
        transforms.RandScaleIntensityd(keys="image", factors=0.1, prob=1.0),
        transforms.RandShiftIntensityd(keys="image", offsets=0.1, prob=1.0),
    ]
)
val_transform = transforms.Compose(
    [
        transforms.LoadImaged(keys=["image", "label"]),
        transforms.ConvertToMultiChannelBasedOnBratsClassesd(keys="label"),
        transforms.NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
    ]
```

```
    )

    train_ds = data.Dataset(data=train_files, transform=train_transform)

    train_loader = data.DataLoader(
        train_ds,
        batch_size=batch_size,
        shuffle=True,
        num_workers=8,
        pin_memory=True,
    )
    val_ds = data.Dataset(data=validation_files, transform=val_transform)
    val_loader = data.DataLoader(
        val_ds,
        batch_size=1,
        shuffle=False,
        num_workers=8,
        pin_memory=True,
    )

    return train_loader, val_loader
data_dir = "./"
json_list = "./brats21_folds.json"
roi = (128, 128, 128)
batch_size = 2
sw_batch_size = 4
fold = 1
infer_overlap = 0.5
max_epochs = 20
val_every = 10
```

```
train_loader, val_loader = get_loader(batch_size, data_dir, json_list, fold, roi)
img_add = os.path.join(data_dir, "TrainingData/BraTS2021_00006/BraTS2021_00006_flair.nii.gz")
label_add = os.path.join(data_dir, "TrainingData/BraTS2021_00006/BraTS2021_00006_seg.nii.gz")
img = nib.load(img_add).get_fdata()
label = nib.load(label_add).get_fdata()
print(f"image shape: {img.shape}, label shape: {label.shape}")
plt.figure("image", (18, 6))
plt.subplot(1, 2, 1)
plt.title("image")
plt.imshow(img[:, :, 78], cmap="gray")
plt.subplot(1, 2, 2)
plt.title("label")
plt.imshow(label[:, :, 78])
plt.show()
os.environ["CUDA_DEVICE_ORDER"] = "PCI_BUS_ID"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
import torch
print(torch.cuda.is_available())

print("cuda 是否可用：", torch.cuda.is_available())
print("cudnn 是否可用：", torch.backends.cudnn.is_available())
print("cuda 的版本为：", torch.version.cuda)
print("torch 的版本为：", torch.__version__)
print("cudnn 的版本为：", torch.backends.cudnn.version())
model = SwinUNETR(
    img_size=roi,
    in_channels=4,
    out_channels=3,
```

```
    feature_size=48,
    drop_rate=0.0,
    attn_drop_rate=0.0,
    dropout_path_rate=0.0,
    use_checkpoint=True,
).to(device)
torch.backends.cudnn.benchmark = True
dice_loss = DiceLoss(to_onehot_y=False, sigmoid=True)
post_sigmoid = Activations(sigmoid=True)
post_pred = AsDiscrete(argmax=False, threshold=0.5)
dice_acc = DiceMetric(include_background=True, reduction=MetricReduction.MEAN_BATCH, get_not_nans=True)
model_inferer = partial(
    sliding_window_inference,
    roi_size=[roi[0], roi[1], roi[2]],
    sw_batch_size=sw_batch_size,
    predictor=model,
    overlap=infer_overlap,
)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=max_epochs)
def train_epoch(model, loader, optimizer, epoch, loss_func):
    model.train()
    start_time = time.time()
    run_loss = AverageMeter()
    for idx, batch_data in enumerate(loader):
        data, target = batch_data["image"].to(device), batch_data["label"].to(device)
        logits = model(data)
```

```
        loss = loss_func(logits, target)
        loss.backward()
        optimizer.step()
        run_loss.update(loss.item(), n=batch_size)
        print(
            "Epoch {}/{} {}/{}".format(epoch, max_epochs, idx, len(loader)),
            "loss: {:.4f}".format(run_loss.avg),
            "time {:.2f}s".format(time.time() - start_time),
        )
        start_time = time.time()
    return run_loss.avg


def val_epoch(
    model,
    loader,
    epoch,
    acc_func,
    model_inferer=None,
    post_sigmoid=None,
    post_pred=None,
):
    model.eval()
    start_time = time.time()
    run_acc = AverageMeter()

    with torch.no_grad():
        for idx, batch_data in enumerate(loader):
            data, target = batch_data["image"].to(device), batch_data["label"].to(device)
            logits = model_inferer(data)
```

```
        val_labels_list = decollate_batch(target)
        val_outputs_list = decollate_batch(logits)
        val_output_convert = [post_pred(post_sigmoid(val_pred_tensor)) for val_pred_tensor in val_outputs_list]
        acc_func.reset()
        acc_func(y_pred=val_output_convert, y=val_labels_list)
        acc, not_nans = acc_func.aggregate()
        run_acc.update(acc.cpu().numpy(), n=not_nans.cpu().numpy())
        dice_tc = run_acc.avg[0]
        dice_wt = run_acc.avg[1]
        dice_et = run_acc.avg[2]
        print(
            "Val {}/{} {}/{}".format(epoch, max_epochs, idx, len(loader)),
            ", dice_tc:",
            dice_tc,
            ", dice_wt:",
            dice_wt,
            ", dice_et:",
            dice_et,
            ", time {:.2f}s".format(time.time() - start_time),
        )
        start_time = time.time()

    return run_acc.avg
def trainer(
    model,
    train_loader,
    val_loader,
    optimizer,
    loss_func,
```

```
    acc_func,
    scheduler,
    model_inferer=None,
    start_epoch=0,
    post_sigmoid=None,
    post_pred=None,
):
    val_acc_max = 0.0
    dices_tc = []
    dices_wt = []
    dices_et = []
    dices_avg = []
    loss_epochs = []
    trains_epoch = []
    for epoch in range(start_epoch, max_epochs):
        print(time.ctime(), "Epoch:", epoch)
        epoch_time = time.time()
        train_loss = train_epoch(
            model,
            train_loader,
            optimizer,
            epoch=epoch,
            loss_func=loss_func,
        )
        print(
            "Final training  {}/{}".format(epoch, max_epochs - 1),
            "loss: {:.4f}".format(train_loss),
            "time {:.2f}s".format(time.time() - epoch_time),
        )

```

```
    if (epoch + 1) % val_every == 0 or epoch == 0:
        loss_epochs.append(train_loss)
        trains_epoch.append(int(epoch))
        epoch_time = time.time()
        val_acc = val_epoch(
            model,
            val_loader,
            epoch=epoch,
            acc_func=acc_func,
            model_inferer=model_inferer,
            post_sigmoid=post_sigmoid,
            post_pred=post_pred,
        )
        dice_tc = val_acc[0]
        dice_wt = val_acc[1]
        dice_et = val_acc[2]
        val_avg_acc = np.mean(val_acc)
        print(
            "Final validation stats {}/{}".format(epoch, max_epochs - 1),
            ", dice_tc:",
            dice_tc,
            ", dice_wt:",
            dice_wt,
            ", dice_et:",
            dice_et,
            ", Dice_Avg:",
            val_avg_acc,
            ", time {:.2f}s".format(time.time() - epoch_time),
        )
        dices_tc.append(dice_tc)
```

```
        dices_wt.append(dice_wt)
        dices_et.append(dice_et)
        dices_avg.append(val_avg_acc)
        if val_avg_acc > val_acc_max:
            print("new best ({:.6f} --> {:.6f}). ".format(val_acc_max, val_avg_acc))
            val_acc_max = val_avg_acc
            save_checkpoint(
                model,
                epoch,
                best_acc=val_acc_max,
            )
        scheduler.step()
    print("Training Finished !, Best Accuracy: ", val_acc_max)
    return (
        val_acc_max,
        dices_tc,
        dices_wt,
        dices_et,
        dices_avg,
        loss_epochs,
        trains_epoch,
    )
start_epoch = 0

(
    val_acc_max,
    dices_tc,
    dices_wt,
    dices_et,
    dices_avg,
```

```
    loss_epochs,
    trains_epoch,
) = trainer(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    loss_func=dice_loss,
    acc_func=dice_acc,
    scheduler=scheduler,
    model_inferer=model_inferer,
    start_epoch=start_epoch,
    post_sigmoid=post_sigmoid,
    post_pred=post_pred,
)
print(f"train completed, best average dice: {val_acc_max:.4f} ")
plt.figure("train", (12, 6))
plt.subplot(1, 2, 1)
plt.title("Epoch Average Loss")
plt.xlabel("epoch")
plt.plot(trains_epoch, loss_epochs, color="red")
plt.subplot(1, 2, 2)
plt.title("Val Mean Dice")
plt.xlabel("epoch")
plt.plot(trains_epoch, dices_avg, color="green")
plt.show()
plt.figure("train", (18, 6))
plt.subplot(1, 3, 1)
plt.title("Val Mean Dice TC")
plt.xlabel("epoch")
```

```
plt.plot(trains_epoch, dices_tc, color="blue")
plt.subplot(1, 3, 2)
plt.title("Val Mean Dice WT")
plt.xlabel("epoch")
plt.plot(trains_epoch, dices_wt, color="brown")
plt.subplot(1, 3, 3)
plt.title("Val Mean Dice ET")
plt.xlabel("epoch")
plt.plot(trains_epoch, dices_et, color="purple")
plt.show()
case_num = "00194"

test_files = [
    {
        "image": [
            os.path.join(
                data_dir,
                "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_flair.nii.gz",
            ),
            os.path.join(
                data_dir,
                "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_t1ce.nii.gz",
            ),
            os.path.join(
                data_dir,
                "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_t1.nii.gz",
            ),
```

```
            os.path.join(
                data_dir,
                "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_t2.nii.gz",
            ),
        ],
        "label": os.path.join(
            data_dir,
            "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_seg.nii.gz",
        ),
    }
]

test_transform = transforms.Compose(
    [
        transforms.LoadImaged(keys=["image", "label"]),
        transforms.ConvertToMultiChannelBasedOnBratsClassesd(keys="label"),
        transforms.NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
    ]
)

test_ds = data.Dataset(data=test_files, transform=test_transform)

test_loader = data.DataLoader(
    test_ds,
    batch_size=1,
    shuffle=False,
    num_workers=8,
```

```
    pin_memory=True,
)
model.load_state_dict(torch.load(os.path.join(root_dir, "model.pt"))["state_dict"])
model.to(device)
model.eval()

model_inferer_test = partial(
    sliding_window_inference,
    roi_size=[roi[0], roi[1], roi[2]],
    sw_batch_size=1,
    predictor=model,
    overlap=0.6,
)


with torch.no_grad():
    for batch_data in test_loader:
        image = batch_data["image"].cuda()
        prob = torch.sigmoid(model_inferer_test(image))
        seg = prob[0].detach().cpu().numpy()
        seg = (seg > 0.5).astype(np.int8)
        seg_out = np.zeros((seg.shape[1], seg.shape[2], seg.shape[3]))
        seg_out[seg[1] == 1] = 2
        seg_out[seg[0] == 1] = 1
        seg_out[seg[2] == 1] = 4
slice_num = 67
img_add = os.path.join(
    data_dir,
    "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_t1ce.nii.gz",
```

```
)
label_add = os.path.join(
    data_dir,
    "TrainingData/BraTS2021_" + case_num + "/BraTS2021_" + case_num + "_seg.nii.gz",
)
img = nib.load(img_add).get_fdata()
label = nib.load(label_add).get_fdata()
plt.figure("image", (18, 6))
plt.subplot(1, 3, 1)
plt.title("image")
plt.imshow(img[:, :, slice_num], cmap="gray")
plt.subplot(1, 3, 2)
plt.title("label")
plt.imshow(label[:, :, slice_num])
plt.subplot(1, 3, 3)
plt.title("segmentation")
plt.imshow(seg_out[:, :, slice_num])
plt.show()
if directory is None:
    shutil.rmtree(root_dir)
```